Software Requirements Specification

for

Autonomous Drone Security Monitor

Version 1.0 approved

Prepared by Andrew Narlock, Roberto Manzano, Tyler Lorentz, James Fischer, Bryan Nguyen Do

TCSS 360 A - Group 18

June 6, 2026

Table of Contents

Table of Contents ..... **i**

Revision History ..... **ii**

1. Introduction **1**

1.1 Purpose 1

1.2 Document Conventions 1

1.3 Intended Audience and Reading Suggestions 1

1.4 Project Scope 1

1.5 References 2

2. Overall Description **2**

2.1 Product Perspective 2

2.2 Product Features 2

2.3 User Classes and Characteristics 3

2.4 Operating Environment 3

2.5 Design and Implementation Constraints 3

2.6 User Documentation 3

2.7 Assumptions and Dependencies 3

3. System Features **4**

3.1 Drone Fleet Simulation 4

3.2 Telemetry Stream 4

3.3 Anomoly Detection 5

3.4 Dashboard Interface 5

3.5 Persistance 5

4. External Interface Requirements **6**

4.1 User Interfaces..... 6

4.2 Hardware Interfaces..... 6

4.3 Software Interfaces..... 6

4.4 Communications Interfaces..... 7

5. Other Nonfunctional Requirements..... 7

5.1 Performance Requirements..... 7

5.2 Safety Requirements..... 7

5.3 Security Requirements..... 8

5.4 Software Quality Attributes..... 8

6. Other Requirements..... 8

Appendix A: Glossary..... 9

Appendix B: Issues List.....9

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and nonfunctional requirements for the Autonomous Drone Security Monitor application. This system is a desktop software solution designed to simulate and monitor a fleet of autonomous drones by processing telemetry data such as position, altitude, battery level, orientation, and velocity. The software will detect abnormal drone behavior, generate operator alerts, and maintain a persistent anomaly log using SQLite. This SRS covers the complete scope of the initial release, including the anomaly detection subsystem, database storage, query features, and graphical dashboard interface developed using the MVC design pattern.

1.2 Document Conventions

Headings and subheadings are in boldface and larger font for readability and structure. Numbered sections are used for organization. Each requirement is considered separate and does not inherit priority from higher-level sections.

1.3 Intended Audience and Reading Suggestions

The intended audience for this document is the developers responsible for building the application along with developers who are curious about the construction of this application. The document can also be useful for researchers and educators who are interested in collecting data on simulated drones.

For developers, starting with the project scope and then moving to the product and system features may be useful when trying to understand what features are implemented and how. Researchers and educators will find more use by reading the project scope to understand what the project entails, then moving to the user documentation in order to learn how to operate the application.

1.4 Project Scope

The Autonomous Drone Security Monitor is a software application designed to simulate and monitor multiple drones in real time. Its purpose is to track telemetry data, detect abnormal behavior such as low battery or unsafe movement, and alert operators through a dashboard interface. The system improves security monitoring efficiency by automating detection, storing anomaly records in SQLite, and providing searchable logs for review.

1.5 References

Oracle. *Java SE 18 Documentation*. Oracle Corporation, 2022. Available at: https://docs.oracle.com/en/java/javase/18/

Hipp, R.D. *SQLite Documentation*. SQLite Consortium. Available at: https://www.sqlite.org/docs.html

2. Overall Description

2.1 Product Perspective

Drones have become an increasingly versatile technology and are improving quickly. It is important that researchers are able to collect data on drone behaviors in order to optimize their performance.

The goal of this project is to develop a new software product. The Autonomous Drone Security Monitor is an independent application that simulates drone telemetry, detects anomalies, stores records in SQLite, and displays information through a graphical dashboard. It is not a replacement for an existing system, but a standalone monitoring solution.

2.2 Product Features

Live telemetry monitoring — displays real-time drone data per cycle including altitude, battery level, velocity, latitude, longitude, and orientation for each drone in the fleet.
Anomaly detection — automatically detects six anomaly types each cycle: critical battery (≤5%), low battery (<15%), crash risk (altitude ≤2m), altitude risk (<5m), GPS spoofing (unexpected coordinate jump), altitude drop (>10m/cycle), and sharp turn (heading change >90°/cycle).
Anomaly severity classification — each detected anomaly is assigned a severity level of INFO, WARNING, or CRITICAL, with a corresponding icon for quick visual identification.
SQLite anomaly persistence — all detected anomalies are written to a local SQLite database each cycle and can be queried by drone ID, anomaly type, or date range.
Audio alerts — plays distinct audio tones for WARNING and CRITICAL anomalies with a cooldown timer to prevent alert fatigue. Alerts can be muted via the Settings menu.
Drone fleet dashboard — a Swing GUI displaying a live fleet list, six telemetry cards for the selected drone, a severity-filtered anomaly log table, and summary statistics (total anomalies, critical count, warning count, average battery).
Grid map visualisation — renders all drone positions on a normalised grid map with directional heading arrows and colour-coded status indicators (green/yellow/red).
Anomaly log filtering — the anomaly table can be filtered by severity (ALL, INFO, WARNING, CRITICAL) in real time.

CSV export — exports the full session anomaly log to a user-specified CSV file via the File menu.
Database query dialog — a dedicated query window accessible from the File menu allowing the user to search stored anomalies by drone ID, type, or date range.
Pause and mute controls — telemetry cycling can be paused and audio alerts muted independently via the Settings menu without closing the application.
Graceful shutdown — on exit the application closes the database connection and shuts down the audio manager cleanly before terminating.

2.3 User Classes and Characteristics

Developers: Developers are responsible for building, maintaining, and updating the system. They are expected to have programming knowledge in Java, database management, and understanding of the MVC design pattern. Developers will have full access to data.

Users: Users are operators or administrators who monitor drone activity through the dashboard. They should have basic computer skills and be able to review alerts, monitor telemetry data, and query anomaly logs. No advanced technical knowledge is required for standard use. Data access may be limited to administrators.

2.4 Operating Environment

The software will operate on standard desktop or laptop computers running Windows 10 or later. The application will be developed in Java and requires the Java Runtime Environment (JRE) or Java Development Kit (JDK). SQLite will be used for database storage. The system should run alongside normal desktop applications without conflicts. Requires the **SQLite JDBC driver** as a runtime dependency alongside the JRE/JDK.

2.5 Design and Implementation Constraints

The system must be developed using the MVC design pattern and implemented in Java. SQLite is required for database storage. The software must run on standard desktop hardware without high memory or processing demands. The interface should remain simple and responsive for real-time monitoring. Developers must follow standard coding practices and maintain compatibility with Windows systems. UI must be updated exclusively on the Event Dispatch Thread (EDT) via SwingUtilities.invokeLater().

2.6 User Documentation

User documentation will include basic user instructions and installation guidelines provided in the github repository for the project.

2.7 Assumptions and Dependencies

Assumptions:

- The users and developers will use JDK for this project.
- The application will be run on a desktop operating system running Windows 10 or later.
- Drone fleets will include a limited number of drones (4).
- Any shared code across team members is version-controlled and accessible via a common repositor

Dependencies:

- ScheduledExecutorService for telemetry loop.
- This project depends on the use of SQLite and its JDBC driver for local database management.

3. System Features

3.1 Drone Fleet Simulation:

3.1.1 Description and Priority

The drone fleet simulation feature is a key component to the project and is required for collecting data on simulated drone behaviors. It is a high priority.

3.1.2 Stimulus/Response Sequences

Stimulus: Data is collected and sent via telemetry stream.

Response: Drone data and user dashboards are updated when data is received from the telemetry stream.

3.1.3 Functional Requirements

Requirement 1: Each drone is represented with the following attributes: GPS position (latitude/longitude), altitude, battery level, orientation, and velocity.

Requirement 2: At least four drones are supported in the simulation.

Requirement 3: Drone simulation data will be sent to the telemetry stream.

3.2 Telemetry Stream:

3.2.1 Description and Priority

The telemetry stream is used to simulate telemetry updates for given time intervals. It tracks simulated drone behavior and is a high priority.

3.2.2 Stimulus/Response Sequences

Stimulus: Data is collected from the drone simulation.

Response: Updated drone data is sent to user dashboards and databases.

3.2.3 Functional Requirements

Requirement 1: Telemetry streams must check and update data on fixed time intervals of 2000ms.

3.3 Anomaly Detection:

3.3.1 Description and Priority

The anomaly detection implements logic for detecting anomalous behavior from a given drone. It is responsible for checking for unsafe maneuvers, battery state risks, and possible GPS spoofing. It is a high priority.

3.3.2 Stimulus/Response Sequences

Stimulus: Data is collected from the telemetry stream that indicates anomalous behaviors.

Response: Information about the anomalous behavior is recorded and sent to a database.

3.3.3 Functional Requirements

Requirement 1: Must check for unsafe maneuvers (sudden altitude drops or sharp turns).

Requirement 2: Checks for low-battery risks.

Requirement 3: Tracks location to check for GPS spoofing.

3.4 Dashboard Interface:

3.4.1 Description and Priority

The dashboard is the user interface for the product and allows users to see real time updates for drone simulation data. It includes a grid map panel showing drone positions with heading arrows, the fleet list panel, the six telemetry cards, and the statistics panel .

3.4.2 Stimulus/Response Sequences

Stimulus: The user interface receives updates from the drone fleet simulator.

Response: The interface is updated with relevant information about the drone behavior.

3.4.3 Functional Requirements

Requirement 1: Updates the user interface in real time to give users a visual representation of the drone behavior.

Requirement 2: Keeps a log of telemetry values and alerts for anomalous behaviors.

Requirement 3: The anomaly log must support real-time severity filtering (ALL, INFO, WARNING, CRITICAL).

3.5 Persistance:

3.5.1 Description and Priority

This feature maintains a record of anomalous behaviors in the drone fleet. It is a high priority.

3.5.2 Stimulus/Response Sequences

Stimulus: An anomalous behavior was detected and an alert was received.

Response: Data on the anomalous behavior is stored in a persistent database.

3.5.3 Functional Requirements

Requirement 1: Records data on anomalous drone behaviors including drone ID, timestamps, anomaly types, and other details.

Requirement 2: Data should be queryable.

4. External Interface Requirements

4.1 User Interfaces

The project will include a single user interface GUI that provides details on different features. The interface includes a main window for viewing real time simulated drone updates, a telemetry stream display for viewing data collected by the simulation, and an anomaly log panel for checking on any anomaly alerts.

A separate window will be implemented for the anomaly data queries.

A menu bar with standard user features will also be implemented. It will include a file menu to save data and exit along with a help menu to answer any frequently asked questions.

4.2 Hardware Interfaces

There are no hardware interfaces as all data is collected from simulated drone fleets.

4.3 Software Interfaces

Java Development Kit (JDK 18+) The application is built and run on JDK 18 or later.

SQLite via JDBC Driver The system interfaces with a local SQLite database file through the SQLite JDBC driver. Anomaly records are written to the database by the anomaly detection subsystem and read back by the query interface.

Java Swing (GUI Library) The dashboard interface is built using Java's built-in GUI library. The interface receives telemetry state updates from the controller layer each simulation tick and renders drone positions, telemetry readouts, and anomaly alerts. User interactions such as query submissions and menu actions are passed back to the controller.

ScheduledExecutorService(Java Concurrency Framework) as a software interface for the telemetry loop.

Operating System (Windows 10+) The application relies on the host OS for file system access and window management.

4.4 Communications Interfaces

The system is not connected to any networks and runs a simulated drone fleet for collecting data. No outside communication interfaces are required.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The product must represent at least four drones along with important data for each drone. It must also provide real time updates to a visual representation of the drone data.

5.2 Safety Requirements

The system must visually report users of potentially unsafe conditions that could damage components or people. These unsafe conditions are denoted by SR - # (safety requirement - #).

SR - 1: High altitude (TBD).

SR - 2: Rapidly changing GPS data/velocity (TBD).

SR - 3: Low battery (battery < 20%, implementation TBD).

SR - 4: Rapidly changing altitude (TBD).

SR - 5: GPS spoofing; impossible movements signifying malicious influence (TBD).

5.3 Security Requirements

The system stores anomaly data in a local SQLite database and is accessible to developers and users. No authentication or encryption is needed as all data is collected from a simulation.

5.4 Software Quality Attributes

Usability: The user interface dashboard should be intuitive and usable for users of varying technical backgrounds.

Reliability: The system should handle simulated telemetry updates continuously without crashing or freezing during a standard monitoring session.

Maintainability: Code should follow the MVC design pattern and standard Java conventions so that any team member can read, modify, or extend any module without difficulty.

Testability: Each feature should be fully testable without requiring the full application to be running.

6. Other Requirements

This project uses an SQLite database and must provide an interface to communicate with this database. The database sorts data by attribute. The interface will allow for queries based on attributes.

Attribute 1: Drone ID

Attribute 2: Timestamps

Attribute 3: Anomaly type.

Attribute 4: Severity Level.

Appendix A: Glossary

Anomaly: An irregular or unexpected drone behavior detected by the system, such as a sudden altitude drop, low battery level, or suspicious GPS location change.

Dashboard: The main graphical interface of the application used by operators to monitor live drone telemetry and view anomaly alerts.

GPS (Global Positioning System): A navigation system used to determine geographic location. In this application, GPS coordinates are simulated.

GUI (Graphical User Interface): A visual interface that allows users to interact with the application through graphical elements such as windows, buttons, and menus.

JDK (Java Development Kit): The software development environment used to build and run the application, version 18 or later.

JRE (Java Runtime Environment): The runtime environment required to execute Java applications.

MVC (Model-View-Controller): A software design pattern that separates the application into three components: the data model, the user interface view, and the controller that manages communication between them.

SQLite: A lightweight relational database used to store and query anomaly records locally on the host machine.

Telemetry: Data collected from a drone during operation, including GPS position, altitude, battery level, orientation, and velocity.